Activity 9

Network Scanner with NMAP

6733172621 Patthadon Phengpinij
Collaborators. Claude (for LaTeX styling and grammar checking)

1 Overview

We will use NMAP ("Network Mapper") to extract various information about host and environment information. Please install nmap. You may install it from the package in your system distribution. Alternatively, you may get it from https://nmap.org/book/install.html. If preferred, you may install the GUI version (zenmap).

2 Exercise

Problem 1. Please connect to your university network (e.g ChulaWIFI or eduroam). We will use a simple ping scan to identify online hosts. eg.

```
$ nmap -v -sn 10.201.3.0/24
$
```

Don't forget to substitute `10.201.3.0` with your network address. Please explain the parameters (-sn).

Solution

What the command does. It performs *host discovery only* over the whole `/24` subnet (256 addresses, `.0`–`.255`) and lists which hosts are online, without scanning any of their ports.

The parameters.

- `-sn` — *"ping scan / no port scan"* (older name `-sP`). It tells nmap to run *only* the host-discovery stage and skip the port-scan stage, so the output is simply the list of live hosts. For discovery nmap does not rely on a plain ICMP echo alone: on the local link, run as root, it sends **ARP requests** (the most reliable on a LAN); to off-link hosts it sends a mix of **ICMP echo**, **ICMP timestamp**, a **TCP SYN to port 443** and a **TCP ACK to port 80**, and marks a host "up" if any probe gets a reply.
- `-v` — *verbose*: prints more detail and reports hosts as they are found (use `-vv` for even more).
- `10.201.3.0/24` — the target in CIDR notation, i.e. the 256-address block `10.201.3.0`–`10.201.3.255`.

Run it as root. ARP discovery and raw packets need privileges; otherwise nmap silently falls back to a less reliable unprivileged method.

```
$ ip -4 addr show eth2 | grep inet          # my address on ChulaWiFi
    inet 10.203.237.69/16 brd 10.203.255.255 scope global eth2
$ sudo nmap -v -sn 10.203.237.0/24
```

Result. Connected to ChulaWiFi, my machine got `10.203.237.69`, so I scanned the `/24` around it, `10.203.237.0/24` (Figure 1). In about 10 s, 2 of 256 addresses were up: my own machine and `10.203.237.236`. The other host's MAC, `02:51:B6:34:83:81`, has the *locally administered* bit set (`02:…`). That means it is a randomized "private" Wi-Fi address, as used by modern phones and laptops, so nmap cannot map it to a vendor (`Unknown`). Only two hosts showed up because the ChulaWiFi subnet is a whole `/16`, so this small `/24` slice holds few of its clients. Across the full `/16` the Problem 3 scan found 373 hosts up.

```
$ sudo nmap -v -sn 10.203.237.0/24
Nmap scan report for 10.203.237.236
Host is up (0.0033s latency).
MAC Address: 02:51:B6:34:83:81 (Unknown)
Nmap scan report for 10.203.237.69
Host is up.
Nmap done: 256 IP addresses (2 hosts up) scanned in 10.06 seconds
```

Figure 1: Ping scan of `10.203.237.0/24` on ChulaWiFi: 2 hosts up.

Problem 2. Pick an ip address from the list in exercise 1 (maybe get the ip address of your friend). We will try to detect the OS from the network footprint. eg.

```
$ nmap -v --osscan-guess 10.201.3.250
$
```

Experiment with several operating systems (Windows, Linux, Mac OS X, iPhone, Android Phone, etc.) Explain how nmap identifies the operating system. You may use -A (Aggressive scan options) to get the result as well.

Solution

The commands (run as root).

```
$ sudo nmap -v --osscan-guess -O 10.203.255.254   # OS detection, guess if no exact match
$ sudo nmap -v -A                10.203.255.254   # aggressive: OS + version + scripts + traceroute
```

How nmap identifies the operating system. It uses **TCP/IP stack fingerprinting**. Every OS implements the TCP/IP stack with slightly different default behaviour, so nmap sends a series of crafted probes (TCP packets to an open and a closed port, a UDP packet to a closed port, and ICMP messages) and examines exactly *how* the host replies. The tell-tale features it measures include:

- initial **TTL** and TCP **window size**;
- the set and **ordering of TCP options** (MSS, window scale, timestamps, SACK, NOPs);
- the **ISN** (initial sequence number) generation pattern and the IP ID sequence;
- the **DF** (don't-fragment) bit, ECN support, and how the host answers unusual flag combinations and ICMP requests.

These values are assembled into a fingerprint and matched against nmap's database (`nmap-os-db`); the closest matches are reported, each with a confidence. Reliable detection needs **at least one open and one closed port**; `--osscan-guess` relaxes the matching so nmap prints its nearest guesses even when there is no perfect match.

What `-A` adds. It bundles **OS detection** (`-O`), **version detection** (`-sV`), **default NSE scripts** (`-sC`) and **traceroute** into one aggressive scan — giving extra corroborating evidence (open-port service banners often name the OS directly, e.g. `Windows` in an SMB banner).

Expected reality for phones. iPhones and Androids (and any firewalled host) are often hard to fingerprint: you will frequently see "`No exact OS matches`" or "`Too many fingerprints match`", and MAC-address randomization hides the vendor. Low accuracy here is a normal result worth noting, not a mistake.

My results. I scanned three targets on ChulaWiFi: the default gateway, an iPad and an Android phone (IPs read from each device's Wi-Fi settings).

Device	Target IP	nmap's OS guess	Notes
Gateway (router)	`10.203.255.254`	None (too many matches)	Up, 1 hop away. MAC OUI `F4:DB:E6` = **Cisco Systems**. All 1000 ports *filtered*.
iPad	`10.203.234.227`	None (host down)	No ARP reply, so nothing to fingerprint.
Android phone	`10.203.97.114`	None (host down)	No ARP reply, so nothing to fingerprint.

```
$ sudo nmap -v -A 10.203.255.254       # gateway
Nmap scan report for 10.203.255.254
Host is up (0.0020s latency).
All 1000 scanned ports on 10.203.255.254 are filtered
MAC Address: F4:DB:E6:5C:C2:FF (Cisco Systems)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

$ sudo nmap -v --osscan-guess -O 10.203.234.227   # iPad
Nmap scan report for 10.203.234.227 [host down]

$ sudo nmap -v --osscan-guess -O 10.203.97.114   # Android
Nmap scan report for 10.203.97.114 [host down]
```

Figure 2: OS detection on ChulaWiFi: the gateway (top) and the two phones (bottom).

Why OS detection failed. The results (Figure 2) show the limits described above:

- **Gateway.** The host is clearly up, but *every* scanned port is filtered: it silently drops the probes and sends no SYN/ACK or RST. Without at least one open and one closed port, nmap cannot build the TCP fingerprint, so many OS signatures fit equally well. The only identifying evidence came from layer 2: the MAC address's OUI shows the device is Cisco hardware, which is typical of campus network gear. That is a good example of evidence outside the TCP/IP fingerprint.
- **iPad and Android.** Both were on-link, so nmap tried ARP. Neither answered, and nmap marked them down in under a second. Likely reasons, all common on campus Wi-Fi: *client isolation* on the access points, which blocks client-to-client traffic; Wi-Fi power saving on an idle phone; and the phones' firewalls. Mobile OSes expose no listening ports by default, so even a reachable phone usually gives a weak fingerprint. Adding `-Pn` skips host discovery, but it cannot help when the network itself drops the frames.

In short, OS fingerprinting only works well against hosts that are reachable and expose a mix of open and closed ports, such as servers and lab machines. Firewalled phones and infrastructure behind a filtering policy do not.

Problem 3. Assuming that you are asked to identify all web servers on a campus, which command will you use to obtain the information. Please explain with an explicit example.

Solution

Idea. A web server is simply a host with an **HTTP/HTTPS port open** (typically `80` and `443`, plus common alternates `8080`/`8443`). So scan the whole campus range for just those ports, keep only the hosts that answer, and use version detection to confirm each one really is a web server (and which one).

Command.

```
$ sudo nmap -T4 -p 80,443,8080,8443 --open -sV --script http-title \
           10.203.0.0/16 -oA campus-webservers
$
```

Breaking it down:

- `-p 80,443,8080,8443` — scan only the web ports (fast across a large range).
- `--open` — report only hosts that actually have one of those ports open, so the output *is* the list of web servers.
- `-sV` — *version detection*: confirms the port speaks HTTP and names the server software (e.g. `Apache httpd 2.4.x`, `nginx`, `Microsoft IIS`).
- `--script http-title` — an NSE script that grabs each site's page title, handy for telling the servers apart.
- `-T4` — the "aggressive" timing template: shorter timeouts and more parallelism, which matters when sweeping tens of thousands of addresses.
- `10.203.0.0/16` — the campus range to sweep (use the real one; `/16` is 65k addresses, so narrow it to the relevant subnets if it is too slow, or add `-Pn` if hosts block ping).
- `-oA campus-webservers` — save results in all formats; the *greppable* file makes it easy to extract just the IPs, e.g.:

```
$ grep -E "80/open|443/open" campus-webservers.gnmap | awk '{print $2}'
```

Result. I ran this against the whole ChulaWiFi range, `10.203.0.0/16` (Figure 3). The sweep took about 14.5 minutes (867 s) and found **373 live hosts**, but **none** had 80, 443, 8080 or 8443 open, so the web-server list is empty. This is the expected answer for this range. `10.203.0.0/16` is the Wi-Fi *client* pool: students' phones and laptops get DHCP addresses there, and with client isolation and host firewalls they do not offer services to each other. The university's web servers live in its server or data-centre subnets and its public address block. To inventory them, you would first find those ranges (from the network team, or from DNS and `whois` for the university's domain) and run the same command on them, with permission from the network administrators. Adding `-Pn` helps there if servers drop ping.

```
$ sudo nmap -T4 -p 80,443,8080,8443 --open -sV --script http-title 10.203.0.0/16 -oA campus-webservers
Nmap done: 65536 IP addresses (373 hosts up) scanned in 866.75 seconds

$ grep -E "(80|443|8080|8443)/open" campus-webservers.gnmap | awk '{print $2}' | wc -l
0
```

Figure 3: Web-server sweep of the ChulaWiFi range `10.203.0.0/16`: 373 hosts up, none serving HTTP(S).

There are several things that can be done with nmap. Please refer to https://nmap.org/book/man.html for more information.

Enjoy!